

INTRODUCTION TO PARALLEL COMPUTING

Parallel hardware has seen an increasing advances in recent decades. Nowadays, personal laptops and desktop use multicore processors. Computing clusters are essential for simulation in Science and Engineering and familiarity with parallel programming and architecture is a must for computer science majors. Real World problems are complex and without parallel computing simulations that are too big are too expensive to carry out. Massively large scale computations, particularly in Climate Modeling, Drug Discovery, and Data Analysis, rely on computer clusters comprising of thousands of processors. From this point view, it is important to understand the fundamentals, tools, and algorithms for parallel programming for currently available platforms. In what follows, an overview of parallel programming models are introduced with an emphasis on design of parallel programs illustrated with examples.

1.1 Serial Computing

Traditionally, codes were written for serial computation, i.e. running a single job at a time. In such case, a problem is divided into a series of instructions, which are consequently executed sequentially on a single processor. In this manner, instructions are execute one at a time. To understand this process further, it is best to first understand the structure of a sequential computer, which consists of main *memory*, a *central processing unit* (CPU), also called processor or core, that are connected via an *interconnection*. A memory consists of a collection of storage locations; both instructions and data are stored in the memory. A memory address points to the location of the memory where the contents, instructions or data, can be accessed. CPU consists of two units: Control Unit and Arithmetic Logic Unit. The Control Unit is for controlling the executing of instructions and

the Arithmetic and Logic Unit execute the instructions. Registers store the data and information about the execution in the CPU. This structure layout is known as a von Neumann machine and this separation of CPU and memory, which stores both program and data, is called the von Neumann bottleneck. This is because in the process of fetching the data and executing the instructions, instructions and data are transferred between the CPU and memory via the interconnect, and therefore the rate at which data can be transferred is limited by the interconnect. As an example, consider the following C code:

```
void add_store (double *a, double *b, double *c) {  
    *c = *a + *b;  
}
```

Here what happens:

- load **a** and **b** from memory to register
- load, combined with an arithmetic addition, to another register
- store back the sum result to memory location of **c**,

where compilers manage memory and registers. The cost of executing this routine includes the cost of the arithmetic operation and the memory accesses, load/store. Data transfers (load/store) have two costs:

- Latency: cost to load or store 1 word
- Bandwidth: average cost per byte to load/store a large chunk.

The performance of an interconnect depends on the latency and the bandwidth. The large chunk is typically more than one floating point number and therefore to increase performance many memory requests must be issued. However, due to latency, it will take time for the data to be transferred. To reach full bandwidth speed, a volume of independent data equal to the bandwidth times the latency has to be in flight, also known as the Little's law:

Concurrency = Bandwidth * Latency.

The overall performance is therefore determined not just by the processor performance, but also the speed of the memory system to transfer data. The clock speed of processors have increased at over years, while memory access times lag behind. In modern computer systems, this bottleneck is addressed by having a hierarchy of successively faster memory devices called caches. Cache is fast, but expensive, memory that takes advantage of the locality of the data. Many programs have a high degree of data locality; for example, when looping over an array, accessing

things nearby previous accesses, called spatial locality, or reusing an item that was previously accessed, called temporal locality. Accessing instructions or data item already residing in cache is cheap, called cache hit. If those not in cache, which is called a cache miss, then there will be a need to access next, slower level of memory. Computer hardware directly manages the cache memory. On-chip caches are faster, but limited in their sizes. Approaches for handling memory access can be viewed as:

- Eliminate memory operations by saving values in small, fast memory (cache or registers) and reusing them.
- Take advantage of better bandwidth by getting a chunk of memory into cache (or vector registers) and using whole chunk.
- Take advantage of better bandwidth by allowing processor to issue multiple loads/stores with a single instruction, or in parallel with other loads/stores/-operations.

Most programs run below maximum performance, mainly due to interconnect. Consider an example of computing inner product, involving one multiplication and addition per cycle. A processor operating at 1 GHz connected to a Dynamic Random-Access Memory (DRAM) with a latency of 100 ns with no caches. Considering two multiply-add units and executing 4 instructions per cycle, the peak performance is 4 GFLOPS. 100 ns latency means that at each cycle, processor waits 100 cycles before executing the next cycle, since each cycle is performed at 1 ns. On this platform, an inner product computation can be done with a speed of 10 MFLOP, which is much smaller than the peak performance. Next we review parallel computing to highlight the need for improving the performance loss on a single processor.

1.2 Parallel Computing Overview

In the previous section, we reviewed the concept of instructions can be executed simultaneously “in flight”, known as instruction-level parallelism, where users have no control over it. On the other hand, users can explicitly schedule more than one instruction to be handled by multiple processors. In the simplest sense, a problem is broken into multiple parts, and consequently further broken down to a series of instructions that can be executed concurrently on different processors. The control and coordination mechanism is employed by the programmer. In the following, we will review the hardware, the programming, and the concepts of this type of parallelism.

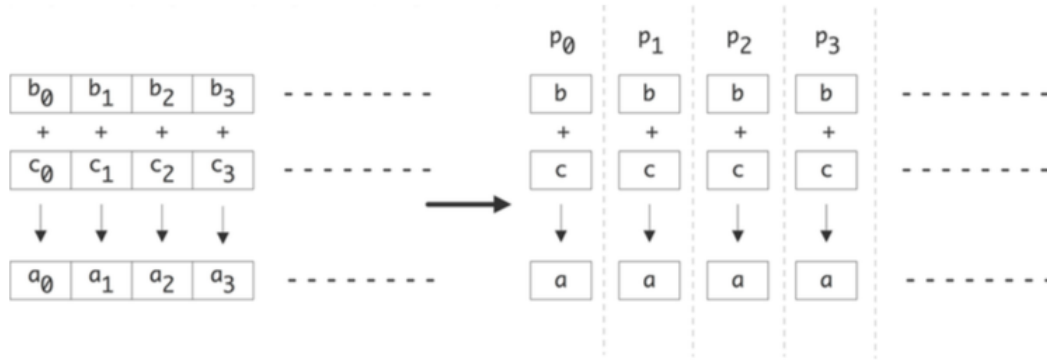
Consider a very simple example of adding two vectors:

```

for (i = 0; i < N; i++)
    a[i] = b[i] + c[i]

```

to be executed on N processors. Ideally, N processors can execute the single instruction $a = b + c$, as depicted below.



Ideally, this parallel algorithm should execute in one unit of time that takes to do one operation and in general, the execution time is linearly proportional to the number of processors. It is however not always simple to parallelize the problem. Often the algorithms must be redesigned to accommodate for parallelization. Even after that, the algorithm may not linearly reduce the execution with the number of processors.

There are a number of ways to classify computer structures:

- Single instruction stream, single data stream, or SISD, where only one instruction stream is being acted on by the processor and only one data stream is loaded at any one cycle.
- Single instruction, multiple data, or SIMD, where all processing units execute the same instruction at any given clock cycle and each processing unit can operate on a different data element.
- Multiple instruction, multiple data (MIMD), where every processor may be executing a different instruction stream and working with a different data stream.

An example of parallelism for SIMD architectures is the vector addition mentioned above, which can be modified as

```

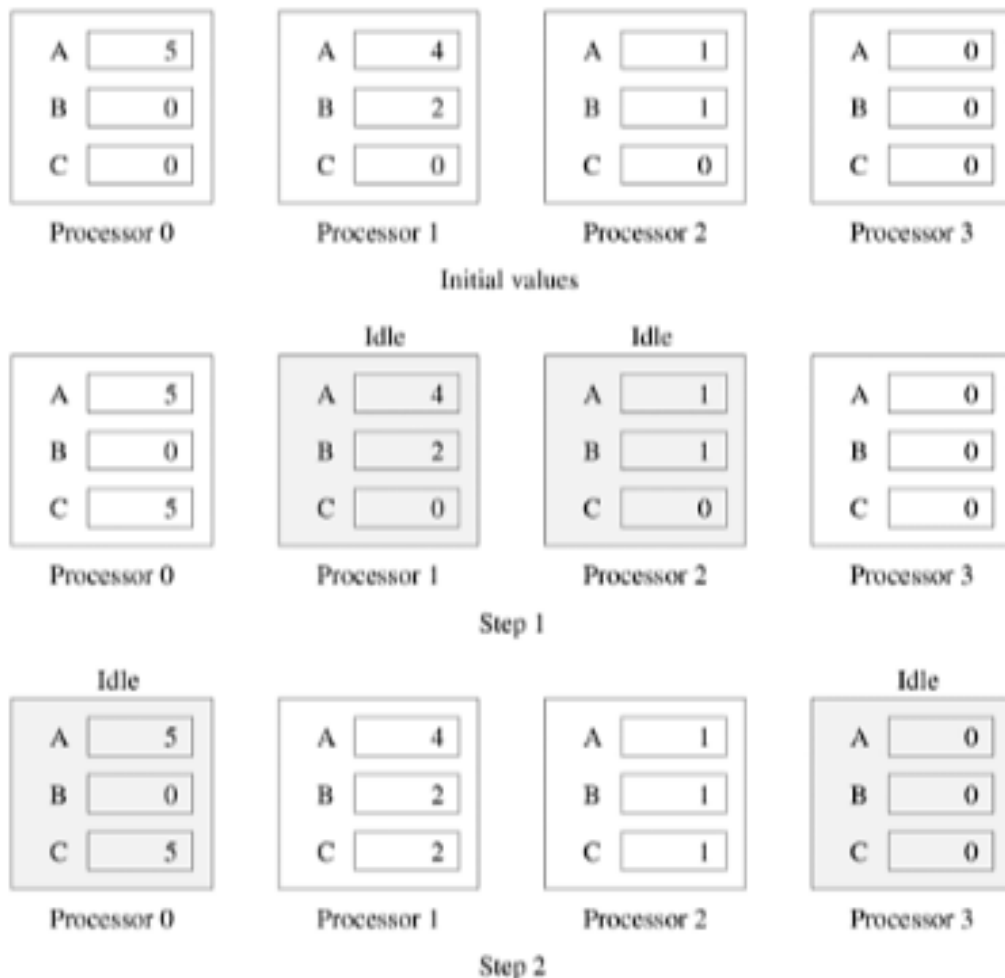
for (i = my_low; i < my_high; i++)
    a[i] = b[i] + c[i]

```

where each processor executes the same addition operation concurrently and syn-

chronously. SIMD architecture relies on the highly structured (regular) nature of the underlying computations, such as the vector addition above, and therefore ideal for parallelizing simple loops operating on huge arrays of data. The example above is called data-parallelism, where data is divided among the processors that each apply the same instructions to their data subsets. While SIMD computers are very efficient on large data parallel problems, they are not suitable for some parallelizations; for example, consider a conditional execution

```
if (B == 0)
    C = A;
else
    C = A/B;
```

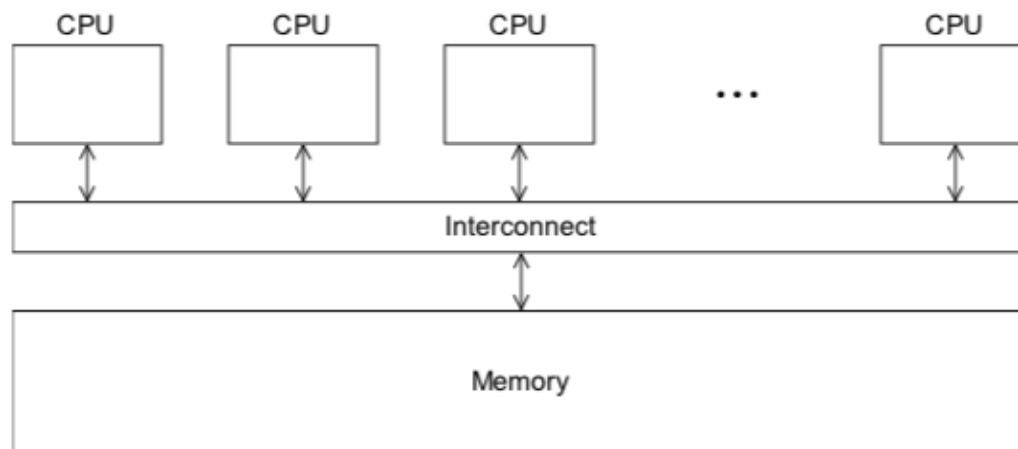


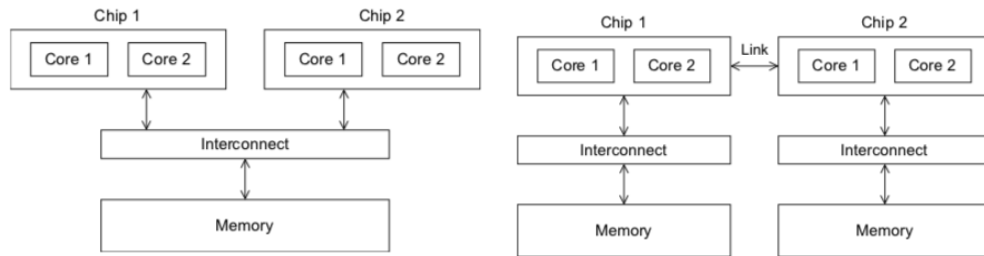
As shown, some processors become idle, leading to poor utilization of resources. Such drawbacks have made SIMD architecture less desirable and nowadays, the

most common parallel architecture is MIMD. We will mainly focus on MIMD machines that operate in Single Program Multiple Data (SPMD) mode, where the same executable act on their copy of the same program simultaneously that will not in general be synchronous as in SIMD machines. The lack of synchronization can be due to processors working on different amounts of data, known as load unbalance, leading to the loss of performance. There are two main types of SPMD models: shared-memory and distributed-memory models.

1.2.1 Shared Memory Model

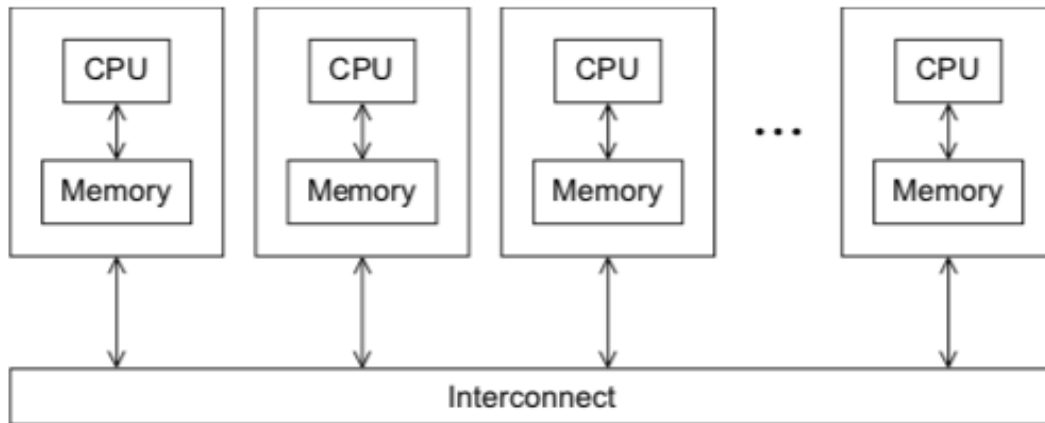
A shared-memory model is a parallel platform of processors that operate autonomously connected to a memory by an interconnection, and each processor has access to all memory locations. If the access time is the same for each processor, the system is called Uniform Memory Access (UMA), most commonly represented Symmetric Multi Processing (SMP) programming model. On the other hand, if the access time is different for certain processes, the system is called Non-Uniform Memory Access (NUMA). NUMA machines are often built by linking SMP machines, where one SMP can directly access memory of another one, however, memory location to which a processor is directly connected can be accessed more quickly than a memory location that must be accessed through the link. If an update to a location in shared memory done by one processor can be visible to all the other processors, then the system is called cache coherent. Below are diagrams of shared memory, UMA, and NUMA, systems:





1.2.2 Distributed Memory Model

Parallel programming is relatively straightforward if all processors have access to any memory location. Moreover, it provides data sharing between processors that is fast and uniform. However, the main disadvantage is the lack of scalability between memory and processors, due to increased traffic on the memory-processor interconnect, and for cache coherent systems, due to the cache-memory management. One solution is to have the processors have their own local memory and therefore each processor operates independently and any updates to its local memory is not visible to other processors. With this setup, the only way one processor can exchange information with another is through a communication network that connects inter-processor memory. This communication is set up by the programmer, as well as synchronization between tasks. Below is a diagram of a distributed memory system. The most widely used distributed memory machines are computer clusters and supercomputers. The interconnection network typically is Ethernet. Often, each computational unit, is a shared memory systems with multiprocessors, linked to other units through communication network. These are called hybrid systems. The individual units can be built from different types of hardware, making the system heterogeneous. Below is diagrams of a distributed memory systems:



1.3 Case study: Matrix Multiplication

Use of performance models to understand performance • Attainable lower bounds on communication • Simple cache model • Warm-up: Matrix-vector multiplication • Naive vs optimized Matrix-Matrix Multiply • Minimizing data movement • Beating $O(n^3)$ operations • Practical optimizations (continued next time)